

AI Coding Agents for *Finance Teams*

A hands-on programme that trains your engineering, quant and analytics teams to direct OpenAI **Codex** and Anthropic **Claude Code** on real financial-software work — safely, and under full review.

24 classes 12 weeks · 2 / week 4–6 hrs / week Live online · mentor-led

Professional certificate

THE PROGRAMME

What this is

Agentic coding tools no longer just autocomplete. They read a repository, plan a change, edit across files, run tests and iterate — under your team's approval. This programme trains your people to **direct the two leading agents** on the software your business actually runs on.

Delivered as a **live online programme**, it recasts every lesson around financial engineering: reconciliation and reporting pipelines, market-data tooling, risk and pricing modules, transaction and positions APIs, and the compliance automation around them. It spans both **general finance** (reporting, reconciliation, dashboards, data pipelines, controls) and **trading, quant and asset management** (algorithmic trading, market data, backtesting, risk and pricing). Every class is lab-driven and ends with a working deliverable in a finance context.

Built for regulated environments. Approvals, OS-level sandboxing, secrets denial, prompt-injection defence and enterprise/managed settings run through the whole course and are consolidated in Class 23 — so agents are adopted with auditability and control, never as a black box.

AUDIENCE

Who this is for

- **Quant & algo developers** and researchers moving faster from idea to tested code
- **Fintech, banking & payments engineers** maintaining and extending production systems
- **Data & analytics engineers** building reporting and reconciliation pipelines
- **Risk, compliance & controls** technologists who need auditable, governed automation
- **Operations & back-office** teams automating repetitive engineering work
- **Tech leads** introducing safe, governed agentic coding to a regulated team

COVERAGE

Tools & stack covered

- **OpenAI Codex** — CLI, Codex Cloud, IDE extension, Codex SDK
- **Anthropic Claude Code** — terminal, VS Code / JetBrains, web, mobile, Claude Agent SDK
- **MCP** (Model Context Protocol) to connect internal data & tools
- **Subagents, hooks & skills** for automation and guardrails
- **OS-level sandboxing** — Seatbelt / bubblewrap / Windows Sandbox
- **GitHub / CI integration** and AI code review at scale

Current model backends taught: Claude Code on **Opus 4.7** (up to xhigh effort, 1M-token context); Codex on **GPT-5.3-Codex** and **GPT-5.5**. The syllabus is **living** — updated as the tools ship new models and features, at no extra cost.

IN PRACTICE

What your team builds with AI agents in finance

Across the programme, participants direct Codex and Claude Code to build, extend and safeguard real financial software — from algorithmic trading to reporting, risk, forecasting and compliance. A snapshot of what they build:

ALGORITHMIC & SYSTEMATIC TRADING

- Strategy, signal and indicator code; feature pipelines
- Order-execution / OMS integration and market-microstructure tools

MARKET DATA & ANALYSIS

- Tick / feed ingestion, normalization and quality checks
- Market analysis and market share / market sizing (TAM) estimation

PORTFOLIO & QUANT RESEARCH

- Portfolio optimization and rebalancing
- Factor models and statistical research notebooks

REPORTING & RECONCILIATION

- P&L and trade-reconciliation fixes and refactors
- Month-end and regulatory report generators

APIS, DATA & PLATFORM

- Transactions / positions APIs and data pipelines
- Reporting-library migrations rolled across a codebase

BACKTESTING & SIMULATION

- Event-driven backtesters and historical replay
- Performance / attribution and Monte-Carlo simulations

RISK & PRICING

- VaR, stress tests, exposure and limit checks
- Derivatives / fixed-income pricing and valuation refactors

FORECASTING & FINANCIAL MODELING

- FP&A, budgeting and cash-flow forecasts
- Scenario and sensitivity analysis

COMPLIANCE, CONTROLS & SURVEILLANCE

- KYC / AML and trade-surveillance automation
- Anomaly / fraud detection

GOVERNANCE & SECURITY

- Hardened permissions + sandbox + secrets for a regulated repo
- MCP links to internal systems: trades / positions DB, market-data API

Customizable syllabus. This is our core curriculum — we shape it to your team. Topics can be added, deepened or swapped to fit your stack, asset classes, internal systems and regulatory context, and we can extend the programme with additional finance modules on request.

Detailed syllabus

Five phases, from first install to a shipped, AI-reviewed capstone. Each class lists the skills your team learns and the finance deliverable they produce.

PHASE 1

Foundations of AI Coding Agents

Understand the landscape, install and configure both agents, and master the approval / permission models before the first real task.

01 The Agentic Coding Landscape — Codex vs Claude Code

YOUR TEAM LEARNS

What agentic coding is — an autonomous loop that reads context, plans, edits, runs commands, verifies and iterates; the full Codex family (CLI, Cloud, IDE, ChatGPT apps, SDK) and Claude Code surfaces (terminal, IDE, web, mobile, Slack, CI); where each wins; and the 2026 model backends.

FINANCE PROJECT

Write a one-page internal brief — "Codex vs Claude Code: where our finance / trading engineering team should use each" — with a six-dimension comparison matrix (latency, review workflow, data-residency & sandbox control, cost, IDE fit, cloud parallelism).

02 Installing & Configuring Claude Code

YOUR TEAM LEARNS

Install across macOS / Linux / Windows; authenticate via subscription or API key; enterprise provider routing (Amazon Bedrock, Google Vertex AI, Azure AI Foundry) for cloud-governed AI; the settings hierarchy (user / project / gitignored-local); model selection; the IDE extension.

FINANCE PROJECT

Install Claude Code and configure a project `.claude/settings.json` inside a real reporting / analytics repo, then run a first scoped task.

03 Installing & Configuring the OpenAI Codex CLI

YOUR TEAM LEARNS

Install the CLI; sign in with ChatGPT or an API key; run `codex doctor`; navigate the interactive TUI; per-project profiles via `~/.codex/config.toml`; model & reasoning selection; native Windows vs WSL2.

FINANCE PROJECT

Create a `config.toml` profile for a finance workspace (pinned model + workspace-write sandbox) and complete a task in it.

04 Approval & Permission Models — the Safety Backbone

YOUR TEAM LEARNS

Why approvals are the safety backbone; Claude Code permission modes (default / acceptEdits / plan / auto) and allow / ask / deny rule syntax; Codex sandbox modes (read-only / workspace-write / danger-full-access) and approval policies; switching live; designing a restrictive default posture.

FINANCE PROJECT

Author a restrictive policy for both agents suited to a regulated environment — deny edits to production config and secret files, block force-push — and prove a disallowed action is denied.

05 Your First Real Engineering Task in Both Agents

YOUR TEAM LEARNS

Scope a real bug precisely; the Claude Code plan → diff → approve → verify loop; run the same task in Codex and compare; read multi-file diffs critically; drive iteration; verify behaviour via app and tests.

FINANCE PROJECT

Fix a real bug in a P&L / trade-reconciliation script and in a market-data ingestion job with each agent, then compare edit quality and effort.

PHASE 2

Your Daily Engineering Workflow

A fast, reliable daily loop: commands, planning, reviewing diffs, running tests and git, teaching the agent your codebase, and recovering safely.

06 Slash Commands & Session Control

YOUR TEAM LEARNS

The core command sets for both TUIs (/help , /clear , /compact , /context , /diff , /review ; Codex /approvals , /status , /plan , /goal , the @ picker); deliberate context management; precise targeting; monitoring cost and limits; recovering a derailed session.

FINANCE PROJECT

Build a team quick-reference of the twelve most-used commands across both tools, with examples drawn from your finance repos.

07 Plan Mode & Goal Mode

YOUR TEAM LEARNS

Plan Mode (read-only research and a proposed plan before any edit) and Goal Mode (works to a defined completion condition) in both agents; writing checkable acceptance criteria; editing the plan to constrain scope; plan-then-execute as the default for risky changes.

FINANCE PROJECT

Use Plan Mode to design a non-trivial change — add pagination + date / instrument filtering to a transactions / positions API, or a new risk metric to a report — with acceptance criteria, then execute the approved plan.

08 Diffs, Multi-File Edits & Review Discipline

YOUR TEAM LEARNS

How agents render diffs across files; reviewing a multi-file feature coherently; accept / reject / request-changes per hunk; inline IDE review; keeping PRs small; asking for rationale; detecting and reverting unintended edits.

FINANCE PROJECT

Have an agent refactor a pricing / valuation module and review every hunk, rejecting anything out of scope.

09 Running Tests, Commands & Git from the Agent

YOUR TEAM LEARNS

Delegating build / run / lint / test; test-driven loops (failing test → implement → pass); git with the agent (staged diffs, clear commit messages, branch hygiene); approving commands with judgment; fixing flaky tests; small frequent commits.

FINANCE PROJECT

TDD a calculation function — e.g. interest accrual or a Value-at-Risk helper — from failing test → implementation → green → commit.

10 Project Memory — CLAUDE.md & AGENTS.md

YOUR TEAM LEARNS

`/init` generates CLAUDE.md (auto-loads each session); the memory hierarchy (managed / user / project / local) and imports; 2026 auto-memory; Codex AGENTS.md (root + nested + personal); a "Review guidelines" section; keeping memory lean and version-controlled for the team.

FINANCE PROJECT

Write production-quality CLAUDE.md and AGENTS.md for a real finance repo, including compliance and code-review guidelines the agents must follow.

11 Checkpoints, Rewind & Session Management

YOUR TEAM LEARNS

Automatic checkpointing and `/rewind` to restore code / conversation / both; the key limitation (checkpoints track file-tool edits, not shell `rm / mv` — git remains the real safety net); resuming, forking and branching sessions; Codex session resume.

FINANCE PROJECT

Run a risky refactor of a reporting module, reject the outcome, and recover cleanly via `/rewind` plus git.

PHASE 3

Extending & Connecting Your Agents

Make the agents fit your stack: connect internal tools via MCP, build skills, deploy subagents, automate with hooks, tune models, and feed visual context.

12 MCP — Connecting Tools, Data & Services

YOUR TEAM LEARNS

What the Model Context Protocol is; adding servers in Claude Code (`http / sse / stdio`; `local / project / user scopes`) and Codex (`config.toml`); MCP resources and prompts; deferred tool loading; and — critically — trust boundaries, OAuth, and prompt-injection risk from MCP content.

FINANCE PROJECT

Connect internal systems via MCP — an issue tracker, a trades / positions database, a market-data API and internal docs — and complete a task using them, with untrusted content treated as hostile.

13 Skills, Custom Commands & Codex Plugins

YOUR TEAM LEARNS

Claude Code Skills and custom commands (`.claude/commands` , `.claude/skills`); frontmatter control (who can invoke, allowed tools, subagent execution); designing a workflow-encapsulating command; Codex profiles and the plugin marketplace; distributing and versioning shared automations.

FINANCE PROJECT

Build a reusable skill — e.g. `/month-end-report` or `/compliance-check` — that encodes a repeatable finance workflow, plus a Codex profile for the project.

14 Subagents — Specialized, Isolated Helpers

YOUR TEAM LEARNS

Subagents run in their own context, tools and permissions; the built-ins (Explore, Plan, General-purpose); defining your own (`.claude/agents/*.md` with tools, model, `permissionMode`); least-privilege design; dispatching parallel subagents; why they protect the main context.

FINANCE PROJECT

Create a read-only code-reviewer subagent scoped to the risk / pricing module and a test-writer scoped to tests, and use them on a real change.

15 Hooks & Workflow Automation

YOUR TEAM LEARNS

Hooks run deterministic code at lifecycle events (`PreToolUse`, `PostToolUse`, `SessionStart`, `Stop`, `PreCompact...`); config via matcher + handler with exit-code 2 to block; practical hooks (auto-format / lint, run tests, inject context, block dangerous commands); enforcing team standards automatically.

FINANCE PROJECT

Implement a `PostToolUse` format + lint hook and a `PreToolUse` guard that blocks force-push and any edit to production config or secret files.

16 Models, Reasoning Effort & Context

YOUR TEAM LEARNS

Claude Code models (Opus 4.7 / Sonnet / Haiku), effort levels (low → xhigh, ultrathink) and 1M-token context for large codebases; Codex models (GPT-5.3-Codex / GPT-5.5) and reasoning levels; cost / performance trade-offs; subagent model overrides; measuring whether more effort actually helped.

FINANCE PROJECT

Benchmark a hard refactor of a legacy finance library at two effort levels and two models; document quality vs cost / latency on a large 1M-context codebase.

17 Multimodal Inputs — Visual Context

YOUR TEAM LEARNS

Why visuals accelerate work; drag-and-drop screenshots / images as context; Codex Appshots (macOS); building UI from a design screenshot; debugging from a screenshot; current limits (image input; no audio / video).

FINANCE PROJECT

Recreate a trading / analytics dashboard from a design screenshot and reconcile it against the spec; diagnose a rendering bug from a screenshot alone.

PHASE 4

Cloud, Collaboration & Scale

Operate beyond one machine: delegate to Codex Cloud, use Claude Code across surfaces, integrate AI code review into GitHub / CI, and orchestrate parallel agents.

18 Codex Cloud — Parallel Tasks & Auto-PRs

YOUR TEAM LEARNS

Codex Cloud runs asynchronous agents in isolated, sandboxed environments; connect GitHub; queue many tasks in parallel, each returning its own PR; configure cloud environments (repo, setup steps, tools, internet-access controls); review cloud PRs; governance for cloud agents.

FINANCE PROJECT

Delegate three parallel Codex Cloud chores across a finance monorepo — with internet access locked down for regulated data — and review / merge the resulting PRs.

19 Claude Code Across Surfaces (Web, Desktop, Mobile, Background)

YOUR TEAM LEARNS

`claude.ai/code` on the web for long-running / parallel tasks even on repos you don't have locally; desktop visual diff review; background agents (`/background` , attach, logs); mobile approvals; teleporting a web session into the terminal; Slack / GitHub triggers.

FINANCE PROJECT

Start a task on the web, hand it to the desktop / terminal, finish it, and run one agent in the background — approving a step from mobile.

20 GitHub Integration & AI Code Review at Scale

YOUR TEAM LEARNS

Connect GitHub apps so both agents open and review PRs; Codex review (`@codex review` , automatic reviews, scoped reviews for security regressions) and `@codex fix` ; Claude Code `/code-review` with `--comment` and `--fix` ; CI automation; acting on AI feedback with rigor.

FINANCE PROJECT

Set up automatic AI code review on a regulated repo, scoped to security regressions, and run a PR through both Codex and Claude Code review.

21 Parallel Agents, Worktrees, /batch & Teams

YOUR TEAM LEARNS

Decompose large work into independent units; git worktrees for isolation; Claude Code `/batch` (split a change into 5–30 units, one background subagent per worktree, each tests + opens a PR); agent teams; when serial is safer than parallel.

FINANCE PROJECT

Use `/batch` (or parallel Codex Cloud tasks) to roll a cross-cutting change — e.g. a reporting-library migration — across the codebase and review each PR.

PHASE 5

Build, Secure & Ship

Operationalize agents: automate with the SDKs, harden security for a regulated team, and deliver a production-grade finance capstone — then certify.

22 Building With the SDKs (Claude Agent SDK & Codex SDK)

YOUR TEAM LEARNS

Headless automation (`claude -p` with text / json / stream-json output; `codex exec` with `--output-schema` for structured JSON); the Claude Agent SDK and Codex SDK; piping and composition; budget / turn controls for safe unattended runs.

FINANCE PROJECT

Build a CI automation that reviews every PR for secret leakage and security issues, emitting structured JSON that gates the merge — a programmatic compliance check.

23 Security, Sandboxing & Governance for Teams

YOUR TEAM LEARNS · FINANCE CENTERPIECE

Permission rules evaluated before any tool runs; OS-level sandboxes (Seatbelt / bubblewrap / Windows Sandbox) with filesystem & network isolation and domain allow / deny; secrets hygiene (denyRead for `~/.ssh` , `~/.aws` , market-data keys; env scrubbing); prompt-injection defence against untrusted web / market / news content; managed / enterprise settings; accountability and audit.

FINANCE PROJECT

Define and test a hardened configuration for a regulated team repo — permissions + OS sandbox + secret denials — and demonstrate a sandbox blocking access outside the allowed scope.

24 Capstone — Ship a Production-Grade Build + Certification

YOUR TEAM LEARNS

Scope a real deliverable; plan with Plan / Goal mode; author CLAUDE.md and AGENTS.md with review guidelines; build with both agents (subagents / hooks / MCP where they add value); quality-gate with tests + AI code review + a clean PR history; ship and document the workflow.

FINANCE PROJECT

Ship a finance deliverable end-to-end — a backtested algorithmic-trading strategy, a reconciliation automation, a risk-&-P&L report generator, a market-data dashboard, or a compliance-check bot — deployed, tested, AI-reviewed and documented, earning the **AI Coding Agents Professional** certificate.

OUTCOMES

What your team can do by the end

- Direct **both Codex and Claude Code** through real financial-software tasks, and choose the right tool per job.
- Ship a **production-grade capstone in a finance context** — a backtested trading strategy, reconciliation automation, risk / P&L report generator, market-data dashboard or compliance-check bot — deployed, tested and AI-reviewed.
- Build at least one **headless SDK automation**, such as a CI compliance / secret-leak gate.
- Integrate **AI code review** into a GitHub / CI workflow, scoped to security regressions.
- Produce a team-ready **"safe agentic coding" governance policy** — permission rules, OS sandboxing, secrets hygiene and prompt-injection defence fit for a regulated environment.
- Earn the Modern Age Coders **"AI Coding Agents Professional"** certificate.

REQUIREMENTS

Prerequisites

DEVICES	A laptop / desktop (Windows, Mac or Linux) with internet and admin rights to install the CLIs.
EXPERIENCE	Helpful but not required — we start from install and ramp quickly; suitable for engineers and technical analysts alike.
ACCOUNTS	A GitHub account, plus access to Claude (Pro / Max) and ChatGPT (Plus / Pro) plans or API keys; enterprise provider routing (Bedrock / Vertex / Azure) covered for teams.
ENVIRONMENT	Works with your existing repos and toolchain; labs can be tailored to your stack on request.

FORMAT & NEXT STEP

Delivery & logistics

FORMAT	Live online cohort — 24 classes, two per week over ~12 weeks; 4–6 hours / week recommended.
DELIVERY	Live, mentor-led and lab-driven; every class ends with a deliverable; capstone review at the end.
LIVING SYLLABUS	Updated continuously as Codex and Claude Code ship new features and models, at no extra cost.
CUSTOMIZATION	Curriculum tailored to your team's stack, priorities and regulatory context; additional finance modules can be added on request.
CERTIFICATION	Modern Age Coders "AI Coding Agents Professional" certificate on completion.

Scope a cohort for your finance team

Team & corporate pricing available on request — tailored to cohort size, schedule and any stack-specific customization.

Modern Age Coders

mahaviracodingclasses@gmail.com
learn.modernagecoders.com

